

1(a) A linked list data structure is designed using object-oriented programming.

The design includes two classes called `node` and `LinkedList`.

The class design for `node` is shown here.

class: <code>node</code>
attributes: <code>private value : integer</code> <code>private nextNode : node</code>
methods: <code>new()</code> <code>getValue()</code> <code>getNextNode()</code> <code>setValue()</code> <code>setNextNode()</code>

`new()` is the constructor method. The constructor method takes a parameter and assigns this to the `value` attribute.

- i. The method `getNextNode()` returns the attribute `nextNode`.

Write the method `getNextNode()` using pseudocode or program code.

[2]

- ii. The method `setValue()` takes an integer as a parameter and assigns this to the attribute `value`.

Write the method `setValue()` using pseudocode or program code.

(b) The class design for `LinkedList` is shown here.

class: <code>LinkedList</code>
attributes: <code>private headNode : node</code>
methods: <code>new()</code> <code>insertNode()</code> <code>outputList()</code>

`new()` is the constructor method. The constructor initialises the attribute `headNode` to a null value.

i. Write the constructor method using pseudocode or program code.

The class design for `node` and `LinkedList` are shown here for your reference.

class: <code>node</code>	class: <code>LinkedList</code>
attributes: <code>private value : integer</code> <code>private nextNode : node</code>	attributes: <code>private headNode : node</code>
methods: <code>new()</code> <code>getValue()</code> <code>getNextNode()</code> <code>setValue()</code> <code>setNextNode()</code>	methods: <code>new()</code> <code>insertNode()</code> <code>outputList()</code>

ii. The method `getValue()` will return the value of a node. The method `getNextNode()` will return the next node to be accessed in the linked list.

The method `outputList()` traverses the linked list starting at the node `headNode`. The method joins the

data in each node and returns it.

Complete the method `outputList()` using pseudocode or program code.

```
function outputList()  
  
    returnValue = ""  
  
    start = .....  
  
    while start ..... null  
        returnValue = returnValue + start. ....() + " "  
        start = start. ....()  
    endwhile  
  
    return .....  
  
endfunction
```

[5]

(c) The method `insertNode()` will insert a new node into the linked list.

The method accepts an integer as a parameter, creates a new node with that data and inserts it into the linked list, e.g. `insertNode(8)` creates a node object with the data 8 and adds it to the linked list.

The program needs to store the following data items in the linked list in the order they are given:

10 25 39

The program then needs to output all of the data in the list using the appropriate method.

Write an algorithm using pseudocode or program code to:

- create a new `LinkedList` object with the identifier `createList`
- insert each of the data items into the linked list (10, 25, 39)
- output the content of the linked list.

[5]

(d) This program uses classes and objects to create a linked list.

An alternative method is creating a 2-dimensional array. In this design the first index stores the data and the second element stores the index of the next data item. Array elements not currently used are also linked together to form a chain of empty values to identify where the next data item can be inserted.

Compare the two different methods of creating a linked list that have been described.

You should include the following in your answer:

- the features of both methods including the reusability of program components

- 2(a) A game is being written that makes use of object-oriented programming. A prototype for one part of the game is being designed that includes a character, a road and a prize to collect.

The road will have 50 spaces that a character can move along. Each space on the road will store a null value or a prize object for the user to collect. Each space is numbered sequentially from the first space (position 0) to the last space (position 49) and will not change during the game. As the player travels down the road, the position the player is on the road will be output.

The road is designed to be a 1-dimensional array with the identifier `road`.

Explain why an array is a suitable data structure to represent the road.

[3]

- (b) The characters and prizes are designed as separate classes. 10 of the spaces on the road will contain an instance of the class Prize. The other spaces will be empty.

The class design for `Prize` is here.

```
class: Prize
attributes:
private name : string
private type : string
private value : integer
methods:
new()
getName()
getType()
getValue()
```

`new()` is the constructor method. The name, type and value are passed to the constructor as parameters which then assigns these to the attributes.

- i. The method `getName()` returns the data in the attribute `name`.
Write the method `getName()` using pseudocode or program code.

[2]

- ii. A global 1-dimensional array, `allPrizes`, stores 10 objects of type `Prize`.

The prize in index 3 has the name "Box", the type is "money" and the value is 25.

Write pseudocode or program code to create a new object for this prize and store it in index 3 of `allPrizes`.

[3]

- iii. The game starts with 10 prizes. Each prize is allocated to one space on the road.

An algorithm needs designing that will generate a random space on the road for each prize. Each road space can only store one prize.

Describe the decisions that will need to be made in this algorithm and how these will affect the program flow.

[3]

- (c) The class design for `Character` is here.

class: <code>Character</code>
attributes: private <code>name</code> : <code>string</code> private <code>money</code> : <code>integer</code>

```
private experience : integer
private roadPosition : integer
```

```
methods:
new()
getName()
getMoney()
getExperience()
getRoadPosition()
changePosition()
updateValues()
```

The four get methods return the associated attribute.

The number of moves is passed to `changePosition()` as a parameter. The method adds this value to the character's position on the road.

The type and value of an object are passed to `updateValues()` as parameters. If the object is money the value is added to the character's money. If the type is experience the value is added to experience. If the type is neither money or experience no changes are made.

- i. `new()` is the constructor method. The name of the character is passed into the constructor as a parameter. The constructor then initialises both the experience and road position of the character to 0 and initialises the amount of money to 5.

Write the constructor method for `Character` using either pseudocode or program code.

You do not need to declare the class, the attributes or any other methods.

- ii. The type and value of a prize are passed as parameters to the method `updateValues`. If the type is money the value is added to the character's money. If the type is experience then the value is added to the experience. If the type is neither money or experience no changes are made.

For example, for the Character `player1`:

`player1.updateValues("money", 10)` updates `player1`'s money by 10

`player1.updateValues("experience", 5)` updates `player1`'s experience by 5

`player1.updateValues("foo", 9)` has no effect on `player1`.

Write pseudocode or program code for the method `updateValues()`.

(d) This incomplete pseudocode algorithm:

- creates a new character with the name Jamal
- loops until the character reaches the end of the road
- generates a random number of spaces to move between 1 and 4 (including 1 and 4)
- moves the character and checks if the new space has a prize
- updates the character attributes if there is a prize
- outputs the character's new attribute values.

Complete the pseudocode algorithm.

```
character1 = new ..... ("Jamal")
newPosition = 0
while newPosition < .....
    move = random(1, 4) //this will generate a random number between 1 and 4
    character1.changePosition(move)
    newPosition = character1.getRoadPosition()
    if newPosition < 50 and road[.....] != null then
        prizeType = road[newPosition].getType()
        valueAmount = road[newPosition].getValue()
        character1.updateValues(....., valueAmount)
        print("Congratulations you are in position", newPosition, "and found",
            road[newPosition].getName())
        print("Money =", character1.getMoney(), "and experience =",
            character1. .... ())
    endif
.....
print("You reached the end of the road")
```

[6]

- (e) The procedure `displayRoad()` outputs the contents of each space in the road. The number of each space is output with either:
- the word “empty” if there is no prize
 - the name of the prize if there is a prize.

```
01 procedure displayRoad()  
02 for x = 0 to 60  
03 print("Space", y)  
04 if road[x] == null then  
05 print("empty")  
06 elseif  
07 print(road[x].getValue())  
08 endif  
09 next x  
10 endprocedure
```

The algorithm contains errors.

Give the line number of **four** different errors and write the corrected line for each error.

Error 1

Error line 1 -----

Correction -----

Error 2

Error line 2 -----

Correction -----

Error 3

Error line 3 -----

Correction -----

Error 4

Error line 4 -----

Correction -----

[4]

- (f) A programmer is going to create a prototype for one small part of the game. Both `road` and `allPrizes` will be needed throughout the whole prototype. The programmer is considering making these global arrays as she thinks it will reduce the development time. Another programmer has suggested that doing this may create some problems when the rest of the game is created at a later stage.

Compare the use of global and local variables in this program.

You should include the following in your answer:

- the use of local and global variables
- alternative methods to using global variables
- the appropriateness of each to this program design.

[illegible]

- 3 OCR DogCare look after dogs while their owners are at work. They use a program developed in an object-oriented paradigm to store details about each dog.

Each dog is declared by using an instance of the class Dog. This has these private attributes:

- name
- breed
- height
- weight

The constructor method sets all the attributes to the values passed in as parameters.

Write pseudocode or program code to define the class Dog. You should include the attributes and constructor method.

You **do not** need to write the set or get methods.

[5]

4(a) A treasure game is being programmed using an object-oriented paradigm.

A class, `Treasure`, is used to store the treasure objects.

The design for the Treasure class, its attributes and methods is shown here.

class: Treasure
attributes: private value : integer private level : string
methods: new() function getValue() function getLevel()

- i. The constructor method takes a value as an integer, e.g. 100, and a level, e.g. "bronze", as parameters and assigns these to the attributes.

Write pseudocode or program code to declare the class `Treasure`.

You should define the attributes and constructor method in your answer.

You do **not** need to write the get methods.

[illegible]

[5]

ii. The get method `getLevel()` will return the appropriate attribute.

Write the method `getLevel()` using either pseudocode or program code.

[2]

iii. Describe the object-oriented programming technique being used in part 9(b)(ii).

[2]

(b) A text-based computer game allows a user to dig for treasure on an island. The island is designed as a grid with 10 rows and 20 columns to store the treasure. Each square is given an x and y coordinate. Some of the squares in the grid store the name of a treasure object. Each treasure object has a value, e.g. 100 and a level, e.g. "Bronze."

A class, Board, is used to store the 10 row (x coordinate) by 20 column (y coordinate) grid.

The design for the Board class, its attributes and methods is shown here.

class: Board
attributes: private grid : Array of Treasure
methods: new() function getGridItem(x, y) function setGridItem(x, y, treasureToInsert)

The constructor initialises each space in the grid to a treasure object with value as -1 and level as an empty string.

Complete the following pseudocode for the constructor method.

```
public procedure new()
for row = ..... to 9
for column = 0 to .....
..... [row, column] = new Treasure(....., "")
next .....
next row
endprocedure
```

[5]

- takes a Board object as a parameter
- accepts the row (x) and column (y) coordinates from the user
- outputs "No treasure" if there is no treasure found at the coordinate (level is an empty string)
- if there is treasure at that coordinate, it outputs the level and the value of the treasure in an appropriate message.

[7]

END OF QUESTION PAPER

Question			Answer/Indicative content	Marks	Guidance
1	a	i	1 mark each to max 2 • Method header getNextNode with no parameters • Returns attribute nextNode e.g. <pre>function getNextNode() return nextNode endfunction</pre>	2	Allow coded solutions. Must return class attribute nextNode, do not allow an input or over-written value <u>Examiner's Comments</u> Many responses did not include a clear function definition, often omitting the word function and writing getNextNode() on its own as given in the question. Pseudocode responses must be clear. Where programming languages are used, candidates should use common conventions and add comments to their code if the meaning might not be clear. While candidates do not have to use the OCR pseudocode it does provide a mechanism for clear and unambiguous solutions to be given.
		ii	1 mark each to max 3 • Method header setValue • ...taking one parameter • Assigning parameter to attribute value e.g. <pre>procedure setValue(newValue) value = newValue endprocedure</pre>	3	Allow coded solutions <u>Examiner's Comments</u> In this question, a common error was to omit a parameter, or to not assign the given parameter to the specific class attribute nextNode.
	b	i	1 mark each • Constructor header taking no parameter • Assigning an appropriate null/none value to headNode e.g. <pre>procedure new() headNode = null endprocedure</pre>	2	Allow coded solutions Appropriate null values do not include integers – the list stores integers. <u>Examiner's Comments</u> Candidates generally found writing the constructor method more challenging than the previous getter and setter methods. The most common error was to pass a value as a parameter to assign to headNode rather than just assigning a null value to it.

Question			Answer/Indicative content	Marks	Guidance
		ii	1 mark for each completed statement to max 5 <pre> function outputList() returnValue = "" start = headNode while start != null returnValue = returnValue + start.getValue() + " " start = start.getNextNode() endwhile return returnValue endfunction </pre>	5	Exact answers only, do not accept incorrectly formed relational operators such as != Instead of != Accept <> instead of != Accept self.__headNode if Python has been used Examiner's Comments Most candidates scored either 4 or 5 marks. There were a significant number of candidates who did not use a valid relational operator for not equal to, using != Instead of !=.
	c		1 mark each to max 5 • Creation of new object of type linkedList assigned to a variable • Using insertNode to correctly insert each node... • ... with all 3 data values inserted in correct order • Calling outputList for the object ... • ...and outputting the return value e.g. <pre> createList = new linkedList() createList.insertNode(10) createList.insertNode(25) createList.insertNode(39) print(createList.outputList()) </pre>	5	Allow createList = linkedList() for instantiation or other valid program syntax Examiner's Comments Many candidates found this question challenging and were given no marks. Some candidates scored 4 or 5 marks. Candidates were less successful in writing a valid instantiation statement to create an instance of linkedList with the identifier createList. Calling the correct method to insertNode to add the required values and then calling the outputList method proved problematic for many candidates. Where candidates had completed all these steps the final requirement to print the value returned by the outputList method was often missed.

Question		Answer/Indicative content	Marks	Guidance
	d	Mark Band 3 – High level (9-12 marks) The candidate demonstrates a thorough knowledge and understanding of OOP and arrays; the material is generally accurate and detailed. The candidate is able to apply their knowledge and understanding directly and consistently to the context provided. Evidence/examples will be explicitly relevant to the explanation. The candidate provides a thorough discussion which is well balanced. Evaluative comments are consistently relevant and well-considered. There is a well-developed line of reasoning which is clear and logically structured. The information presented is relevant and substantiated. Mark Band 2 – Mid level (5-8 marks) The candidate demonstrates reasonable knowledge and understanding of OOP and arrays; the material is generally accurate but at times underdeveloped. The candidate is able to apply their knowledge and understanding directly to the context provided although one or two opportunities are missed. Evidence/examples are for the most part implicitly relevant to the explanation. The candidate provides a reasonable discussion, the majority of which is focused. Evaluative comments are, for the most part appropriate, although one or two opportunities for development are missed. <i>There is a line of reasoning presented with some structure. The information presented is in the most part relevant and supported by some evidence.</i> Mark Band 1 – Low Level (1-4 marks) The candidate demonstrates a basic knowledge of OOP and arrays with limited understanding shown; the material is basic and contains some inaccuracies. The	12	Answers may include, but are not limited to, some of the points below: AO1: Knowledge • OOP allows any number of elements • ...new object created each time • ...do not need to declare number of elements • Array need to know number of elements • OOP can be reused in other programs for other lists without having to make significant changes • Array would need more changes for other programs such as size of array • Array needs a way to indicate which elements have been used and which are empty AO2: Application • OOP allow use of dynamic memory • ...can work for any number of elements • ...do not need to know before hand • Array has static memory allocation • ...need to know maximum number • ...memory wastage if not all used • ...can run out of memory • Array can be more complicated to assign new elements as use of free list is needed to identify/track where data can be stored AO3: Evaluation • Allow either approach as long as justified E.g. OOP is more appropriate because a linked list would only need to be created once as a class and can then be used in other programs with small adaptations. It is a more efficient use of memory because a node is created when needed and deleted when not needed so space in memory does not need to be reserved. <u>Examiner's Comments</u> The final question required candidate to discuss technical content. Many candidates did not appreciate how the alternative implementational

Question			Answer/Indicative content	Marks	Guidance
			candidates makes a limited attempt to apply acquired knowledge and understanding to the context provided. The candidate provides a limited discussion which is narrow in focus. Judgements if made are weak and unsubstantiated. <i>The information is basic and communicated in an unstructured way. The information is supported by limited evidence and the relationship to the evidence may not be clear.</i> 0 marks No attempt to answer the question or response is not worthy of credit.		approaches worked. This may demonstrate limited practical programming experience. Some candidates listed features of the OOP paradigm without specific linkage to the scenario. Very few Level 3 responses were seen.
			Total	29	

Question			Answer/Indicative content	Marks	Guidance
2	a		1 mark each to max 3. Max 2 for generic answers with no relation to scenario. e.g. • Has a set/fixed number of values • ...and the number of spaces in the road will not change • Stores data of one type • ... as the array is only made up of prize objects • Stores data linearly • ... match the linear nature of the road • Array contents are mutable • ... so prizes can be added/removed from the road • A single identifier is used to directly index • ... any position in the road • Can be iterated by index • ... to perform an operation on all road positions	3	<u>Examiner's Comments</u> Many responses were too vague, showing little knowledge of the properties of arrays. Relatively few candidates appeared to be able to make explicit links to the scenario to achieve full marks.
	b	i	1 mark each • Function/subroutine with identifier getName taking no parameters • Returning name e.g. <pre> public function def getName() getName(self): return name return self.__name endfunction public function getName(){ getName() { return name return this.name } } </pre>	2	BP1 Do not award procedure or method BP1 Allow self as an additional parameter if Python is used. BP1 If an access modifier is given for the method, it must be public and not private. BP2 Do not allow any modified name attribute to be returned. <u>Examiner's Comments</u> While many candidates had little difficulty giving code for a getter () there were a number of common errors. Some candidates used a private access modifier when a getter () needs to be public. There was often erroneous use of 'procedure' whereas a getter () is a function that must return a value. Some candidates tried to set values within the getter () function when it should only have returned the class attribute value.

Question			Answer/Indicative content	Marks	Guidance
		ii	1 mark each • New instance of prize ... • ... with "Box", "money" and 25 as parameters • Assigned to allPrizes index 3 e.g. allPrizes[3] = new prize("Box", "money", 25) allPrizes[3] = prize.new("Box", "money", 25) allPrizes[3] = prize("Box", "money", 25)	3	MP2 allow any order of parameters "Box" and "Money" must be strings and 25 must be an integer Allow prize.new() as new is given as the constructor method in the class diagram <u>Examiner's Comments</u> Many candidates struggled with the instantiation of an object. Where candidates made an attempt to instantiate some did not use a string for "box" and "money" or did not give 25 as an integer but instead gave the string "25".
		iii	1 mark for each bullet to maximum 3 e.g. • Decision - check whether the space already has a prize allocated ... • Action if true - another space/number will need to be generated • Action if false - the prize will be stored here • Decision - check if all 10 prizes have been allocated ... • Action if true - the algorithm needs to stop generating numbers • Action if false - a new number/space needs to be generated and checked	3	Give: • 1 mark for stating a decision • 1 mark for the action required if true • 1 mark for the action required if false <u>Examiner's Comments</u> There were only two reasonable decisions that could be given from the scenario details. Candidates needed to make it clear that a decision with a Boolean output was present that would dictate two potential outcomes. Some candidates quoted actions such as 'randomly assign space for prize' which did not represent a decision. Many responses described the mechanics of setting up the game and the random spaces but did not highlight the program conditions/decisions as required.

Question			Answer/Indicative content	Marks	Guidance
	c	i	1 mark each • Constructor header (any suitable name e.g. new, constructor, create, init) • ...taking one parameter only • Initialising name to the parameter • Initialising money to 5 • Initialising experience to 0 and roadPosition to 0 e.g. Pseudocode <pre> public procedure example: new(pName) name = pName experience = 0 roadPosition = 0 money = 5 endprocedure </pre> Python <pre> def __init__(self, Example: pName): self.__name = pName self.__experience = 0 self.__roadPositio n = 0 self.__money = 5 </pre> C# <pre> public Example: Character(string pName) { string name = pName; int experience = 0; int roadPosition = 0; int money = 5; } </pre>	5	Allow minor changes to identifiers as long as purpose is clear. Allow procedure new(pName) this.name = pName ... (or similar e.g. self.name) Allow two parameters if one is <i>self</i> and the response is clearly in Python. The parameter name should be different to the attribute name. <u>Examiner's Comments</u> It was clear that those candidates with limited OOP programming knowledge found the writing of a relatively simple constructor method difficult. Those with relevant programming experience often found this to be a very straightforward question. Common errors included passing additional values to set the experience, roadPosition and money attributes rather than setting them to the constant values indicated in the question.

Question			Answer/Indicative content	Marks	Guidance
		ii	1 mark each • Procedure/method header ... • ... taking two parameters, type (or similar) followed by value (or similar) • ... • ... compare type parameter with "money" • ... compare type parameter with "experience" • ... both attributes updated correctly and nothing else modified e.g. <pre> public procedure updateValues(pType, pValue) if pType == "money" then money = money + pValue elseif pType == "experience" experience = experience + pValue endif endprocedure def updateValues(self, pType, pValue): if pType == "money": money += pValue elif pType == "experience": experience += pValue </pre>	5	Do not allow Function for BP1 BP2 parameters must be given in the correct order to match the calls to updateValues() in the question. "money" and "experience" must be string values <u>Examiner's Comments</u> The updateValues procedure again proved problematic for candidates with limited OOP experience. No marks were given for the first mark point if a function was declared as there was no return value. Parameter names needed to be fit for purpose, understandable, and had to match the order given in the question scenario to work for the given example calls.

Question		Answer/Indicative content	Marks	Guidance
	d	1 mark for each completed space <pre> character1 = new Character("Jamal") newPosition = 0 while newPosition < 50 move = random(1, 4) character1.changePosition(m ove) newPosition = character1.getRoadPosition() if newPosition < 50 and road[newPosition] != null then prizeType = road[newPosition].getType() valueAmount = road[newPosition].getValue() c haracter 1.updateValues(prizeType, valueAmount) print("Congratulations you are in position", newPosition, "and found", road[newPosition].getName()) print("Money", character1.getMoney(), "and experience", character1.getExperience()) endif endwhile print("You reached the end of the road") </pre>	6	Allow road.length // len(road) instead of 50 Allow <=49 instead of < 50 <u>Examiner's Comments</u> Nearly all candidates achieved some marks, and a majority scored five or six marks.
	e	1 mark each • (Line 02) for x = 0 to 49 • (Line 03) print("Space", x) • (Line 06) else // elseif road[x] <> null • (Line 07) print(road[x].getName())	4	Line 07 allow print(road[x].name) <u>Examiner's Comments</u> Many candidates scored three or four marks but in general candidates found it harder to identify errors in the code than to complete code in the previous question. Some candidates didn't give the line number but rewrote the incorrect line before giving the corrected line, which was acceptable, although not ideal given the scaffolding.

Question		Answer/Indicative content	Marks	Guidance
	f	Mark Band 3 – High level (7-9 marks) The candidate demonstrates a thorough knowledge and understanding of global variables and the alternatives; the material is generally accurate and detailed. The candidate is able to apply their knowledge and understanding directly and consistently to the context provided. Evidence/examples will be explicitly relevant to the explanation. <i>There is a well-developed line of reasoning which is clear and logically structured. The information presented is relevant and substantiated.</i> Mark Band 2 – Mid level (4-6 marks) The candidate demonstrates reasonable knowledge and understanding of global variables and the alternatives; the material is generally accurate but at times underdeveloped. The candidate is able to apply their knowledge and understanding directly to the context provided although one or two opportunities are missed. Evidence/examples are for the most part implicitly relevant to the explanation. The candidate provides a reasonable discussion, the majority of which is focused. Evaluative comments are, for the most part appropriate, although one or two opportunities for development are missed. <i>There is a line of reasoning presented with some structure. The information presented is in the most part relevant and supported by some evidence.</i> Mark Band 1 – Low Level (1-3 marks) The candidate demonstrates a basic knowledge of global variables and the alternatives with limited understanding shown; the material is basic and contains some inaccuracies. The candidates makes a limited attempt to apply acquired knowledge and understanding to the context provided. The candidate provides a limited discussion which is narrow in focus. Judgements if made are weak and unsubstantiated. <i>The information is basic and communicated in an unstructured way. The information is supported by limited evidence and the relationship to the evidence may not be</i>	9	AO1: Knowledge and Understanding Indicative content - Global variables are created when the program starts, all subroutines can access/update the contents - Local variables are created in the subroutine they are created in, they are not accessible directly from any other subroutine - Local variables are removed from memory when the subroutine ends. - Local variables can be passed as parameters to a function to be updated, and then returned to override the original local variable - Local variables can be passed by reference to a subroutine to allow the content of the variable to be updated AO2: Application - The variables will be stored in memory throughout the whole code execution. However, the amount of data they are storing is relatively low so would not use a lot of memory. - When the game is expanded, the amount of data may increase so it could be memory intensive, especially if graphics are used in the game. - Both arrays are needed throughout the whole game so keeping them as global will make writing the code easier as the programmer will not need to keep passing them as parameters and setting return values. - Only one part of the game is being created initially and therefore the use of global variables would not affect the efficiency greatly. However, when the program expands, it could cause accuracy / testing / debugging and maintenance problems. AO3: Evaluation - As this is only a prototype, the use of global variables would be beneficial. - However, when the game expands, the use of global variables could create issues such as running out of memory, coupling, testing & debugging problems and maintenance problems.

Question			Answer/Indicative content	Marks	Guidance
			<i>clear.</i> 0 mark No attempt to answer the question or response is not worthy of credit.		The programmer may be best to keep the variables as local and then pass them between the different subroutines as parameters byVal and byRef. <u>Examiner's Comments</u> Most responses were Level 2 for definitions and some expansion to passing parameters. Very few candidates were able to go into depth about alternatives to global variables such as passing by value and passing by reference in detail or extending to issues such as scalability within a larger more extended game. Few candidates picked up on the fact that this was a more limited prototype that was likely to be expanded on which would require more consideration to be given to variable scope.
			Total	40	

Question	Answer/Indicative content	Marks	Guidance
3	1 mark per bullet to max 5: • Suitable logic for class dog declaration • Suitable logic to define the 4 (private) attributes: name, breed, height, weight • Suitable logic to declare a <u>public</u> method for constructor (e.g. new or class name)... •taking <u>only</u> 4 different parameters in any order. • Suitable logic to set the values of each attribute MP4 - if it is obvious the candidate has used Pythonic syntax: allow the additional 5th parameter representing the object e.g. self, turtle <u>but this must be the first parameter listed.</u> MP2 - If they have done the above BP2 can be considered implicitly met (Python doesn't require attributes to be declared outside the constructor)	5	Mark in a vertical line against each MP. Ignore data types in attribute names Allow colon/empty brackets at the end of class def Example Solutions <u>Pseudocode style</u> <pre>class Dog private name private breed private height private weight public procedure new (nameIn, breedIn, heightIn, weightIn) name = nameIn breed = breedIn height = heightIn weight = weightIn endclass</pre> <u>Java / C# Style</u> <pre>Class Dog { private String name private String breed private float height private float weight public void Dog(String name, String breed, float height, float weight) { this.name = name this.breed = breed this.height = height this.weight = weight } }</pre> <u>Python Style</u> <pre>class Dog: def __init__(self, name, breed, height, weight): self.name = name self.breed = breed self.height = height self.weight = weight</pre> <u>Examiner's Comments</u>

Question			Answer/Indicative content	Marks	Guidance
					There were a range of responses to this question with the most common responses being written in pseudocode or python. There was a clear division between candidates who were comfortable writing Object Oriented Programming (OOP) code and class constructors and those that were not. Less successful candidates with little OOP experience often confused the class declaration with the constructor declaration. A few input the values rather than setting the attributes to the values passed in the parameters.
			Total	5	

Question			Answer/Indicative content	Marks	Guidance
4	a	i	1 mark each - Defining class Treasure - Defining the private attributes value and level - Defining a new public procedure... ...Taking two parameters (integer and string) - Correctly assigning both parameters to the attributes e.g. <pre>class Treasure private value private level public procedure new(valueP, levelP) value = valueP level = levelP endprocedure endclass</pre>	5	Allow use of this/self or equivalent dependent on language <pre>public procedure new(value, level) this.value = value this.level = level endprocedure</pre> Python answers must either use comments to indicate private attributes or use the double underscore private attribute convention to be credited. <pre>self.__level self.level # private</pre> <u>Examiner's Comments</u> This question either tended to be very poorly answered or very well answered. There was a clear division between candidates who were comfortable writing OOP code and class constructors and those that were not. Weaker candidates with little OOP experience often confused the class declaration with the instantiation method or offered no response. Many candidates often assigned the parameters to the private attribute values rather than setting the attributes to the parameters being passed in. Where candidates gave responses in programming languages and the intent of the response was clear, marks were given. However, sometimes Python programmers did not make it clear (by convention/comment) that class attributes were private.

Question			Answer/Indicative content	Marks	Guidance
		ii	1 mark each • get level method header with no parameter • Returning level attribute e.g. public function getLevel() return level endfunction	2	Note Python <i>self</i> will appear, but no other parameters def getLevel(self): <u>Examiner's Comments</u> While there were many good responses it was also noted that a significant number of candidates erroneously tried to send a parameter to a getter() function and then return the same parameter. Some candidates did not return a value or simply tried to print the value. Other errors included writing the getter method as a function call by omitting any indication that a function was being defined.
		iii	1 mark each • Encapsulation • Allowing an attribute to only be accessed/changed via a method	2	<u>Examiner's Comments</u> Some candidates randomly picked an erroneous OOP term like polymorphism. Those candidates who could correctly identify the use of encapsulation did not always go on to specifically state that this meant access was controlled by a public getter() method.
	b		1 mark for each completed statement public procedure new() for row = 0 to 9 for column = 0 to 19 grid[row, column] = new Treasure(-1,"") next column next row endprocedure	5	<u>Examiner's Comments</u> Most candidates scored at least the first 2 mark-points for the grid dimensions, but far fewer could construct OOP code requiring the attribute and the relevant default parameter value to be given.
	c		1 mark each to max 7 • Procedure declaration taking parameter • Taking two inputs for row and column from the user • Accessing item at grid position... • ...using correct get methods getGridItem • Checking (treasure) object's level/value... • ...using correct get method getLevel getValue • ...outputting "No treasure" if empty • ...otherwise outputting value and level e.g. procedure guessGrid(gameboard) rCoord = input("Enter R coordinate")	7	Note candidates may attempt to access private attributes directly gameboard.grid(x,y) for example, instead of gameboard.getGridItem(x,y). Credit cannot be given for the dependent second mark using appropriate get method if they do this, but FT marks can be awarded for later points if a reasonable attempt has been made. <u>Examiner's Comments</u> Many candidate responses gave very little beyond the procedure declaration and taking in the x and y coordinates as inputs, thus scoring 2 marks at most. There was less successful understanding of how to use the get() methods provided in the

Question			Answer/Indicative content	Marks	Guidance
			<pre> cCoord = input("Enter C coordinate") treasureItem = gameBoard.getGridItem(rCoord, cCoord) if treasureItem.getLevel() == "" then print("No treasure") else print("This treasure is level ", treasureItem.getLevel(), " with value ", treasureItem.getValue()) endprocedure </pre>		scenario to access the data. This is an area of the specification that candidates require extensive practical experience of to be able to fluently answer questions like this in examination conditions. Many candidates tried to used grid[x,y] by inserting the coordinates into the parameter values and did not identify it as an instance of an object that needed its attributes to be accessed via the appropriate getter methods. Those candidates who tried to access the attributes directly without the appropriate getter methods did achieve some further marks with follow through marks. Competent coders often gave responses worthy of full marks within just a few lines of code. Exemplar 3 <pre> def guessGrid(Board): def guessGrid(Board): x = int(input("enter x")) y = int(input("enter y")) place = Board.getGridItem(x,y) if place.getLevel() == "": print("No treasure") else: print("Treasure found! Valued at", place.getValue()) print("it is a", place.getLevel()) </pre> This response clearly shows a logical and well-structured algorithm that uses the appropriate get() methods to access the relevant attributes of the passed parameter object. Candidates who are fluent in the use of OOP can present elegant and concise solutions.
			Total	21	